

ECUACIONES DE RECURRENCIA E INVESTIGACIÓN DE ALGORITMO DIVIDE Y VENCERÁS

1. solucion

Ecuación 1

$$T(n) = 2T(n/2) + O(1)$$

Aplicando el Teorema Maestro:

$$a = 2, b = 2 \text{ y } f(n) = O(1).$$

Calculamos:

$$n^{(\log_2 2)} = n$$

Como $O(1)$ crece más lentamente que n , se obtiene:

$$T(n) = O(n)$$

Resultado: Complejidad lineal.

Ecuación 2

$$T(n) = 4T(n/2) + O(1)$$

Aplicando el Teorema Maestro:

$$a = 4, b = 2 \text{ y } f(n) = O(1).$$

Calculamos:

$$n^{(\log_2 4)} = n^2$$

Como $O(1)$ crece más lentamente que n^2 , se obtiene:

$$T(n) = O(n^2)$$

Resultado: Complejidad cuadrática.

Ecuación 3

$$T(n) = 2T(n/2) + O(n^2)$$

Aplicando el Teorema Maestro:

$$a = 2, b = 2 \text{ y } f(n) = O(n^2).$$

Calculamos:

$$n^{(\log_2 2)} = n$$

Como n^2 crece más rápidamente que n , se obtiene:

$$T(n) = O(n^2)$$

Resultado: Complejidad cuadrática.

Ecuación 4

$$T(n) = T(n-1) + O(1)$$

En esta ecuación el tamaño del problema disminuye en una unidad en cada llamada:

$$T(n) = T(n-1) + 1$$

$$T(n) = T(n-2) + 2$$

$$T(n) = T(n-3) + 3$$

El proceso continúa hasta llegar al caso base. Debido a que se realizan aproximadamente n llamadas, la complejidad es:

$$T(n) = O(n)$$

Resultado: Complejidad lineal.

Ecuación 5

$$T(n) = T(n/3) + T(2n/3) + O(1)$$

El problema se divide en dos partes, una de tamaño $n/3$ y otra de tamaño $2n/3$.

Como:

$$n/3 + 2n/3 = n$$

El árbol de recurrencia tiene aproximadamente $\log(n)$ niveles y cada nivel representa un trabajo total proporcional a n .

Por lo tanto:

$$T(n) = O(n \log n)$$

Resultado: Complejidad $O(n \log n)$.

Ecuación 6

$$T(n) = 3T(n/4) + O(n^2)$$

Aplicando el Teorema Maestro:

$$a = 3, b = 4 \text{ y } f(n) = O(n^2).$$

Calculamos:

$$n^{(\log_4 3)} \approx n^{0,792}$$

Como n^2 crece más rápidamente que $n^{0,792}$, se obtiene:

$$T(n) = O(n^2)$$

Resultado: Complejidad cuadrática.

2. Resumen de resultados

1. $T(n) = 2T(n/2) + O(1) \rightarrow O(n)$
2. $T(n) = 4T(n/2) + O(1) \rightarrow O(n^2)$
3. $T(n) = 2T(n/2) + O(n^2) \rightarrow O(n^2)$
4. $T(n) = T(n-1) + O(1) \rightarrow O(n)$
5. $T(n) = T(n/3) + T(2n/3) + O(1) \rightarrow O(n \log n)$
6. $T(n) = 3T(n/4) + O(n^2) \rightarrow O(n^2)$

3. Investigación de Algoritmo Divide y Vencerás

Merge Sort

Merge Sort es un algoritmo de ordenamiento que utiliza el paradigma de Divide y Vencerás. Su objetivo es ordenar una lista de elementos de manera eficiente.

Funcionamiento

Se divide en tres etapas:

1. Dividir

La lista original se divide en dos partes aproximadamente iguales. Cada una de estas partes vuelve a dividirse hasta obtener listas de un solo elemento.

Por ejemplo:

[8, 3, 5, 1, 4, 2, 7, 6]

Se divide en:

[8, 3, 5, 1] [4, 2, 7, 6]

Y posteriormente:

[8, 3] [5, 1] [4, 2] [7, 6]

Hasta llegar a:

[8] [3] [5] [1] [4] [2] [7] [6]

2. Vencer

Cuando cada parte contiene un solo elemento, se considera que ya está ordenada. Luego, las partes se van ordenando recursivamente.

Por ejemplo:

[8] + [3] → [3, 8]

[5] + [1] → [1, 5]

[4] + [2] → [2, 4]

[7] + [6] → [6, 7]

3. Combinar

Finalmente, las partes ordenadas se combinan para formar una lista completamente ordenada.

[3, 8] + [1, 5] → [1, 3, 5, 8]

[2, 4] + [6, 7] → [2, 4, 6, 7]

Resultado final:

[1, 2, 3, 4, 5, 6, 7, 8]

Ecuación de recurrencia

La ecuación de recurrencia de Merge Sort es:

$$T(n) = 2T(n/2) + O(n)$$

Esto se debe a que:

- **$2T(n/2)$** : el problema se divide en dos subproblemas de tamaño $n/2$.
- **$O(n)$** : combinar las dos partes ordenadas requiere recorrer los elementos.

Aplicando el Teorema Maestro:

$$T(n) = O(n \log n)$$

Por lo tanto, la complejidad temporal de Merge Sort es **$O(n \log n)$** .

Conclusión

Merge Sort es un ejemplo del paradigma Divide y Vencerás porque divide un problema grande en problemas más pequeños, resuelve cada uno de ellos de forma recursiva y finalmente combina sus resultados. Su

complejidad temporal es $O(n \log n)$, lo que lo convierte en un algoritmo eficiente para ordenar grandes cantidades de datos.